

Contents

Documentación de Casos de Uso — Aliflow	3
Cambios de esta revisión (28-jul-2026, decisiones de Negocios)	5
Actores	6
UC1 — Iniciar sesión institucional	6
UC2 — Recargar saldo	7
UC3 — Consultar menú del día	7
UC4 — Comprar almuerzo	8
UC5 — Retirar y entregar almuerzo	8
UC6 — Iniciar sesión operativa	9
UC7 — Configurar integración con sistema externo	9
UC8 — Administrar menú	10
UC9 — Consultar métricas y operación	10
UC10 — Consultar estado de sincronización	10
UC11 — Consultar detalle de venta (soporte re-emisión de comprobante)	11
UC12 — Gestionar usuarios del local	11
Cartilla de fidelidad — UC13, UC14, UC15	12
UC13 — Consultar cartilla de fidelidad	12
UC14 — Configurar programa de fidelidad del local	13
UC15 — Canjear premio de la cartilla	13
Acta del 30-jul-2026 — UC16 y UC17	14
UC16 — Administrar inventario reservado para Aliflow	14
UC17 — Configurar horario máximo de retiro	14
Super-Admin de Aliflow — UC18, UC19 y UC20	15
UC18 — Dar de alta un local y crear su vista de proveedor	15
UC19 — Brindar soporte a los locales	15
UC20 — Administrar la plataforma	15
Casos de uso incluidos/extendidos (sub-flujos reutilizables)	16
Documentación del Diagrama de Clases — Aliflow	17
1. Paquete “Usuarios”	17
2. Paquete “Proveedor y Menú”	18
InventarioReservado — agregado el 30-jul-2026	19
3. Paquete “Wallet y Pagos”	19
Qué cambió en las clases	20
Por qué se eliminó el patrón Strategy, y no se conservó “por las dudas”	20
Lo que esta decisión resolvió	20
Lo que esta decisión costó	21
Lo que no cambió	21
4. Paquete “Órdenes”	21
5. Paquete “Fidelidad” (requisito nuevo, 28-jul-2026)	22
6. Paquete “Integración con ERP externo”	23
Principios SOLID aplicados	24

Patrones de diseño usados (y por qué, no solo “porque sí”)	25
Malos olores evitados	25
Supuestos y pendientes de este diagrama (a validar con el equipo/Negocios)	25
Cambios aplicados en la revisión del 28-jul-2026 (decisiones de Negocios)	26
Correcciones aplicadas en esta revisión (27-jul-2026)	27
Documentación de Diagramas de Objetos — Aliflow	27
1. Billetera y orden de un estudiante	28
2. Integración multi-tenant con los ERP de dos locales reales (Barú/Contífico y Caramel Coffee/Alpwin)	29
Documentación del Diagrama de Componentes — Aliflow	30
Componentes principales	30
Cliente	30
Aliflow API (FastAPI)	31
Capa de Integración	31
Procesamiento asíncrono	31
Sistemas externos	31
Decisiones de este diagrama que quedan abiertas	32
Documentación del Diagrama de Despliegue — Aliflow	32
Nodos	33
Flujos de comunicación relevantes	33
Decisiones que quedan abiertas en este diagrama	34

Documentación de Casos de Uso – Aliflow

Diagrama fuente: ../../uml/casos-de-uso.puml (mismo directorio) — es la fuente editable; ../../uml/casos-de-uso.svg es la imagen renderizada que se muestra arriba (generada con el servidor público de PlantUML). Si editas el .puml, regenera el SVG para que la imagen quede sincronizada — instrucciones en ../../uml/README-diagramas.md.

Referencia: cada caso de uso indica el paso correspondiente de ../../Flujos-Aliflow-Revision.html (formato {rol}-{número}, ver nota en ../../Hallazgos-Ingenieria-API-Generica.md sección 5) del que se derivó.

Convención visual: fondo amarillo = propuesta de Ingeniería sin validar con Negocios. Misma convención usada en ../../uml/diagrama-clases.puml y ../../uml/diagrama-componentes.puml. En esta revisión (28-jul-2026) los tres casos de uso que estaban en amarillo (UC12/UC13/UC14, del rol Administrador) fueron eliminados o reasignados. Los que están en amarillo ahora son otros: **UC13, UC14, UC15 y “Acreditar sello”**, del requisito nuevo de cartilla de fidelidad. Los números UC13/UC14 se reutilizaron y **no tienen relación con los anteriores**.

Cambios de esta revisión (28-jul-2026, decisiones de Negocios)

Qué cambió	Antes	Ahora
Número de roles	4 actores primarios (Estudiante, Proveedor, Operador, Administrador de plataforma)	3 — Estudiante, Proveedor, Operador. El “Administrador” es el Proveedor: el gerente del local. No existe un super-admin de plataforma.
UC12/UC13/UC14	Propuesta de Ingeniería para el super-admin (alta de proveedores, métricas globales, gestión de usuarios)	UC13 y UC14 eliminados . UC12 se redefine como “Gestionar usuarios del local” y pasa al Proveedor.
Personal múltiple por local	Pendiente de definir	Confirmado: un local puede tener varias cuentas de Proveedor y varias de Operador.
ERP del proveedor	Un solo ERP objetivo (se creía que Barú usaba Alpwin)	Multi-ERP: cada local usa el suyo (Barú → Contífico, Caramel Coffee → Alpwin), con conexión bidireccional .
Código de retiro (UC5)	Pendiente entre QR y numérico; propuesta de UUID firmado	Código numérico corto (6 dígitos), dictado de viva voz al Operador.
Recarga de saldo (UC2)	Pendiente entre saldo por proveedor y saldo único	Recarga única , distribuida internamente por Aliflow.
Cartilla de fidelidad	No existía como requisito	Requisito nuevo: el estudiante acumula sellos por compra y al completar la cartilla gana un premio. Se agregan UC13, UC14, UC15 y el sub-flujo UC5d. Cuántos sellos y qué premio siguen sin definir.

Actores

Actor	Tipo	Descripción
Estudiante	Primario	Usuario institucional que recarga saldo, consulta el menú, compra y retira almuerzos.
Proveedor	Primario	La persona que administra un local: menú, inventario, métricas, integración con su ERP y las cuentas de su propio personal. Es el gerente del local — Negocios confirmó (28-jul-2026) que el rol “Administrador” y el rol “Proveedor” son el mismo. Un local puede tener varias cuentas de Proveedor.
Operador	Primario	Valida las compras realizadas por los estudiantes y marca la entrega física del almuerzo en el punto de entrega. Un local puede tener varios. (En discusiones previas del equipo aparecía como “cajero”; el nombre oficial del rol es Operador .)
Proveedor de Identidad (Google OAuth)	Secundario / sistema externo	Autentica al Estudiante mediante OAuth 2.0 / OpenID Connect.
Sistema ERP del local	Secundario / sistema externo	No es uno solo: cada local de la universidad usa un ERP distinto (Barú → Contífico , Caramel Coffee → Alpwin , etc.). La conexión es bidireccional — Aliflow lee inventario/menú del ERP y le devuelve órdenes y pagos. Todos entran por la misma API genérica (patrón Adapter, ver ../../Hallazgos-Ingenieria-API-Generica.md sección 3).

Nota sobre vocabulario: “Proveedor” se usa en dos sentidos que conviene no confundir. Como **actor/rol** es la persona que gerencia el local. Como **entidad** (clase Proveedor del diagrama de clases) es el local/negocio en sí — el tenant. En el diagrama de clases la persona se modela como Administrador, subclase de UsuarioProveedor; ver ../02-Modelamiento-Parte-Estatica/b-Diagrama-de-Clases.md, sección “Usuarios”.

UC1 — Iniciar sesión institucional

Campo	Detalle
Actor primario	Estudiante
Actor secundario	Proveedor de Identidad (Google OAuth)

Campo	Detalle
Referencia	est -1 – Autenticación
Precondición	El estudiante posee un correo institucional válido en el dominio autorizado.
Flujo principal	1. El estudiante selecciona “Iniciar sesión con Google”.2. El sistema redirige al proveedor de identidad y solicita consentimiento (<<include>> UC1a).3. El backend valida que el dominio del correo pertenezca a la lista institucional autorizada y que el token no esté expirado.4. Se genera una sesión autenticada (token/JWT).
Flujo alternativo (extend)	UC1b – Crear perfil y tarjeta virtual: si es el primer ingreso del estudiante, se crea su perfil y tarjeta virtual con saldo en cero antes de continuar.
Excepción	Si el dominio del correo no pertenece a la lista institucional autorizada, se rechaza el acceso (validación crítica marcada en el flujo).
Postcondición	El estudiante queda autenticado con una sesión válida.

UC2 – Recargar saldo

Campo	Detalle
Actor primario	Estudiante
Referencia	est -2 – Recarga de tarjeta virtual
Precondición	El estudiante está autenticado (UC1).
Flujo principal	1. El estudiante selecciona “Recargar saldo” e ingresa un monto (o elige uno predefinido).2. Se redirige al método de recarga definido por el negocio.3. Al confirmarse el pago, el sistema actualiza el saldo (operación atómica) e incluye la generación del comprobante (<<include>> UC2a).4. El estudiante ve su saldo actualizado.
Postcondición	El saldo del estudiante queda incrementado; existe un comprobante de recarga asociado.
Resuelto (28-jul-2026)	El estudiante hace una única recarga , que va a una bolsa común gastable en cualquier local; Aliflow distribuye internamente hacia cada proveedor. Ya no hay recarga separada por local. Lo único abierto es <i>cuándo</i> ocurre esa distribución interna – ver ../.../Decisiones-Pendientes-Negocios.md, punto 4.

UC3 – Consultar menú del día

Campo	Detalle
Actor primario	Estudiante
Actor secundario	Sistema ERP del Proveedor
Referencia	est -3 – Consulta del menú del día

Campo	Detalle
Precondición	El estudiante está autenticado.
Flujo principal	1. El estudiante accede a “Menú del día”.2. El sistema sincroniza/consulta el catálogo del proveedor (<<include>> UC3a), vía la API genérica o una base local ya sincronizada.3. Se muestra el menú con disponibilidad actualizada.
Excepción	Si no hay sincronización en tiempo real, puede presentarse el caso “vendido antes de confirmar” al momento de la compra (ver UC4).
Postcondición	El estudiante visualiza platos disponibles, precio y stock.

UC4 – Comprar almuerzo

Campo	Detalle
Actor primario	Estudiante
Actor secundario	Sistema ERP del Proveedor
Referencia	est -4 – Compra del almuerzo
Precondición	El estudiante consultó el menú (UC3) y tiene saldo disponible.
Flujo principal	1. El estudiante selecciona un plato y confirma la compra.2. El sistema valida saldo y disponibilidad (<<include>> UC4a, con revalidación de stock justo antes de confirmar).3. Si ambas validaciones pasan: se descuenta el saldo (atómico), se crea la orden en estado “Comprado”, se genera el comprobante (<<include>> UC4b) y se notifica al proveedor (<<include>> UC4c).
Flujo alternativo	Si falla alguna validación (saldo o stock insuficiente), se muestra el error y no se ejecuta ningún descuento.
Excepción crítica	Concurrencia: dos estudiantes no deben poder comprar la última unidad simultáneamente – requiere bloqueo/transacción atómica (ya identificado como riesgo).
Postcondición	Orden creada en estado “Comprado”; saldo y stock descontados; comprobante generado; ERP del proveedor notificado.

UC5 – Retirar y entregar almuerzo

Campo	Detalle
Actor primario	Estudiante y Operador (caso de uso compartido – ambos participan de la misma transacción)
Referencia	est -6 (Estudiante) y op -2/op -3/op -4/op -5 (Operador)
Precondición	Existe una orden en estado “Comprado” (UC4).
Flujo principal	1. El estudiante se presenta en el punto de entrega con su código de validación.2. El operador busca la orden (<<include>> UC5a – por código o búsqueda manual por nombre/ID institucional si el estudi-

Campo	Detalle
	ante no tiene el código).3. El sistema valida y confirma la entrega (<<include>> UC5b): verifica que el estado sea “Comprado” (no “Entregado”), cambia el estado a “Entregado”, registra timestamp y operador, e invalida el código (uso único).
Flujo alternativo (extend)	UC5c – Manejar excepción de entrega: código inválido/ ya usado (mensaje de error con hora de entrega previa), o fallo de conexión (v1 no tiene modo offline — riesgo ya documentado).
Postcondición	Orden en estado “Entregado”; código invalidado.
Resuelto (28-jul-2026)	Formato del código: numérico corto de 6 dígitos . El estudiante se lo presenta/dicta al Operador, que lo digita en Aliflow para marcar el retiro. Desbloquea el prototipo de mockups.
Pendiente	Regla de expiración de una orden nunca retirada (sección 5.3 del hallazgo de Ingeniería).

UC6 – Iniciar sesión operativa

Campo	Detalle
Actor primario	Proveedor, Operador
Referencia	prov - 1, op - 1
Precondición	El usuario tiene una cuenta con el rol correspondiente ya asignada (no es OAuth institucional, son credenciales propias del rol).
Flujo principal	1. El usuario inicia sesión con sus credenciales de rol.2. El sistema valida el rol y da acceso a la interfaz correspondiente (panel de administración para Proveedor; interfaz simplificada optimizada para uso rápido en el caso de Operador).
Postcondición	Sesión autenticada con permisos del rol correspondiente.

UC7 – Configurar integración con sistema externo

Campo	Detalle
Actor primario	Proveedor (junto con Ingeniería)
Actor secundario	Sistema ERP del Proveedor
Referencia	prov - 2
Precondición	El local está dado de alta en Aliflow; su ERP (Contífico, Alpwin, Odoo u otro) ya fue identificado.
Flujo principal	1. Se especifica el endpoint/mecanismo de conexión del ERP.2. Se define el mapeo de datos (productos, precios, stock) al modelo canónico de Aliflow.3. La API genérica (patrón Adapter) actúa como capa de abstracción, sin que el resto de Aliflow dependa de la implementación específica del ERP.

Campo	Detalle
Postcondición	El adaptador correspondiente (ContificoAdapter, AlpwinAdapter, OdooAdapter, ...) queda configurado y operativo para ese local, en ambos sentidos: Aliflow lee su inventario y le devuelve órdenes y pagos.
Nota de arquitectura	Este caso de uso es la materialización directa del reto técnico investigado en ../.../Hallazgos-Ingenieria-API-Generica.md — ver secciones 3 y 4.2 (demo validado con Odoo Community).

UC8 — Administrar menú

Campo	Detalle
Actor primario	Proveedor
Referencia	prov-3
Precondición	El proveedor está autenticado (UC6).
Flujo principal	1. El proveedor define el menú del día (plato, descripción, precio, stock inicial), manualmente o sincronizado desde su ERP.2. El sistema valida datos mínimos (precio > 0, nombre no vacío, stock ≥ 0) antes de publicar.
Postcondición	Menú publicado y disponible para consulta (UC3).

UC9 — Consultar métricas y operación

Campo	Detalle
Actor primario	Proveedor
Referencia	prov-5
Precondición	El proveedor está autenticado.
Flujo principal	1. El proveedor accede a su panel de métricas.2. El sistema calcula y muestra: almuerzos vendidos por día/semana, ingresos, platos más vendidos, y órdenes “Comprado” pendientes de “Entregado”, a partir del histórico de órdenes de Aliflow.
Postcondición	El proveedor visualiza el estado operativo de su negocio.
Pendiente	Modelo de cobro de Aliflow al proveedor (comisión/suscripción) — decisión abierta, sección 5.2 del hallazgo de Ingeniería.

UC10 — Consultar estado de sincronización

Campo	Detalle
Actor primario	Proveedor
Referencia	prov-4
Precondición	El proveedor tiene su integración configurada (UC7).
Flujo principal	1. El proveedor consulta el estado de

Campo	Detalle
	sincronización de inventario en su panel.2. El sistema muestra la última comunicación exitosa con su ERP y eventos pendientes/fallidos (tabla de reconciliación, ver arquitectura outbox en ../../Hallazgos-Ingenieria-API-Generica.md sección 3.4).
Postcondición	El proveedor conoce si hay ventas pendientes de reflejarse en su ERP externo.

UC11 – Consultar detalle de venta (soporte re-emisión de comprobante)

Campo	Detalle
Actor primario	Proveedor
Actor secundario	Sistema ERP del Proveedor
Referencia	prov -6
Precondición	Existen ventas registradas en Aliflow (UC4).
Flujo principal	1. El proveedor consulta o descarga el detalle de una venta (monto, plato, estudiante, fecha/hora) desde Aliflow.2. El proveedor re-emite la factura válida en su propio sistema contable/ERP, usando ese detalle como fuente de datos.
Postcondición	El proveedor cuenta con la información necesaria para emitir su factura fiscal real; Aliflow nunca emite un comprobante con validez tributaria.
Nota importante	Este es el caso de uso donde se detectó la contradicción entre el Acta (25-jun-2026) y el flujo documentado respecto al comprobante tributario – ver ../../Hallazgos-Ingenieria-API-Generica.md sección 5.1. El modelo correcto es el aquí descrito: re-emisión por el proveedor, no emisión fiscal directa de Aliflow.

UC12 – Gestionar usuarios del local

Campo	Detalle
Actor primario	Proveedor (administrador del local)
Referencia	prov -1 – “el proveedor o personal autorizado ”. Este caso de uso cierra ese vacío.
Precondición	El proveedor está autenticado (UC6).
Flujo principal	1. El proveedor crea una cuenta nueva para su local, eligiendo el rol: Proveedor (otro administrador) u Operador .2. Si es Operador, lo asocia a un punto de entrega.3. Puede revocar accesos o restablecer credenciales de las cuentas de su propio local.
Postcondición	El personal del local queda habilitado con el rol que le corresponde, con acceso limitado a ese local.

Campo	Detalle
Regla de alcance	Un Proveedor solo puede gestionar cuentas de su propio local . No hay ningún rol con visibilidad sobre todos los locales.

Cambio del 28-jul-2026: este caso de uso reemplaza al antiguo UC14 (“Gestionar usuarios y roles”, del super-admin). Los antiguos **UC12** (“Dar de alta / gestionar proveedores”) y **UC13** (“Métricas globales de la plataforma”) quedaron **eliminados** junto con el rol Administrador de plataforma que los justificaba. Ambos eran hipótesis de Ingeniería y estaban marcados en amarillo justamente por eso.

Consecuencia abierta: si nadie dentro del sistema da de alta un local nuevo, ese paso es **manual y fuera del alcance de v1** — lo hace el equipo de Aliflow directamente contra la base de datos, junto con la configuración de la integración (UC7). Está registrado como punto abierto en ../.../Decisiones-Pendientes-Negocios.md.

Cartilla de fidelidad — UC13, UC14, UC15

⚠ **Requisito nuevo (28-jul-2026), modelado como propuesta de Ingeniería.** Negocios pidió una “cartilla de fidelidad”: el estudiante acumula un sello por compra y al completar la cartilla gana un premio. **Cuántos sellos hacen falta y cuál es el premio todavía están en definición**, así que se modelan como *configuración por local* (UC14) y no como constantes del sistema — cuando Negocios los defina, es un valor en base de datos, no un cambio de código.

Nota sobre la numeración: UC13 y UC14 existieron antes con otro significado (métricas globales y gestión de usuarios del super-admin) y fueron eliminados en esta misma revisión. Estos son casos de uso nuevos que reutilizan esos números.

UC13 — Consultar cartilla de fidelidad

Campo	Detalle
Actor primario	Estudiante
Referencia	Ninguna aún — requisito nuevo, no está en el flujo ni en el acta.
Precondición	El estudiante está autenticado (UC1).
Flujo principal	1. El estudiante abre su sección de fidelidad.2. El sistema muestra una cartilla por local en el que haya comprado: sellos acumulados sobre sellos requeridos, premio ofrecido, y fecha de expiración si aplica.3. Si alguna cartilla está COMPLETA , se destaca que tiene un premio disponible para canjear (UC15).
Postcondición	El estudiante conoce su avance en cada local.

UC14 — Configurar programa de fidelidad del local

Campo	Detalle
Actor primario	Proveedor (administrador del local)
Referencia	Ninguna aún — requisito nuevo.
Precondición	El proveedor está autenticado (UC6).
Flujo principal	1. El proveedor activa o desactiva el programa de fidelidad de su local.2. Define cuántos sellos requiere la cartilla, en qué consiste el premio, cuántos sellos como máximo se pueden acumular por día, y si la cartilla caduca.3. El sistema valida los datos mínimos (sellos requeridos > 0).
Postcondición	El programa queda activo; a partir de ahí, cada entrega acredita sellos (UC5d).
Regla de alcance	El programa es por local , no de la plataforma: el costo del premio lo absorbe el local, así que cada uno define el suyo. Un local puede no tener programa.

UC15 — Canjear premio de la cartilla

Campo	Detalle
Actor primario	Estudiante y Operador (transacción compartida, igual que UC5)
Referencia	Ninguna aún — requisito nuevo.
Precondición	El estudiante tiene una cartilla en estado COMPLETA en ese local.
Flujo principal	1. El estudiante elige el plato del premio y confirma el canje.2. El sistema crea una orden real (<<include>> UC4) con <code>esCanje = true</code> y total \$0: se descuenta el stock y se genera código de retiro como en cualquier compra, pero no se descuenta saldo.3. La cartilla pasa a CANJEADA mediante una actualización atómica y condicional (<code>WHERE estado = 'COMPLETA'</code>), igual que la redención del código de retiro.4. El Operador entrega el premio y confirma, como en UC5.
Flujo alternativo	Si la cartilla ya fue canjeada entre el paso 1 y el 3 (doble canje concurrente), la actualización afecta 0 filas y se informa error sin crear la orden.
Postcondición	Cartilla en CANJEADA; orden de canje entregada; inventario del local descontado.
Pendiente	Cómo debe representarse una venta de \$0 en el ERP del local (documento de cortesía / descuento 100%) — se resuelve distinto en Contífico que en Alpwin. Ver <code>../.../Decisiones-Pendientes-Negocios.md</code> , punto 9.

Acta del 30-jul-2026 — UC16 y UC17

UC16 — Administrar inventario reservado para Aliflow

Campo	Detalle
Actor primario	Proveedor
Referencia	Acta del 30-jul-2026, sección 1.3.
Precondición	El proveedor está autenticado (UC6) y tiene platos publicados (UC8).
Flujo principal	1. El proveedor consulta el cupo actualmente asignado a Aliflow por plato.2. Asigna, aumenta o reduce las unidades destinadas exclusivamente a las ventas por Aliflow.3. El sistema valida que el cupo no sea negativo ni menor al ya consumido.4. El cupo queda disponible para la compra de estudiantes (UC4).
Postcondición	Aliflow puede vender hasta el cupo asignado, con independencia de lo que ocurra en la caja del local.
Regla de negocio	Aliflow valida la compra contra este cupo, no contra el stock total del ERP. Si el local tiene 100 almuerzos y asigna 25 a Aliflow, la aplicación deja de vender al llegar a 25 aunque el ERP reporte unidades libres.
Por qué se diseñó así	Elimina la sobreventa por diseño en lugar de mitigarla sincronizando más seguido. Aliflow deja de competir con la caja por el mismo contador. Ver ../ ../ /Decisiones-Pendientes-Negocios.md, punto 10.
Pendiente	El proceso de asignación y su visualización en el panel quedaron como tarea para la próxima reunión.

UC17 — Configurar horario máximo de retiro

Campo	Detalle
Actor primario	Proveedor
Referencia	Acta del 30-jul-2026, secciones 6.1 y 6.2.
Precondición	El proveedor está autenticado (UC6).
Flujo principal	1. El proveedor define la hora máxima hasta la que se pueden retirar los almuerzos de su local.2. El sistema la guarda en <code>Proveedor.horaMaximaRetiro</code> .3. A partir de ahí, el mensaje de confirmación que ve el estudiante al comprar se arma con ese valor, y la expiración del código de retiro se calcula con él.
Postcondición	El horario aplica solo a ese local y se refleja automáticamente en la app del estudiante.
Regla de negocio	El horario no puede estar fijo en el código. Cada local define el suyo. La vigencia del código termina, como máximo, al terminar el día de la compra.

Super-Admin de Aliflow – UC18, UC19 y UC20

⚠ **Acordado verbalmente en la reunión del 30-jul-2026; no consta en el acta.** Conviene incorporarlo al acta para que quede constancia formal.

Nota sobre el vaivén: el 28-jul Negocios indicó que este rol **no existía** y sus casos de uso se eliminaron (ver la tabla de cambios al inicio de este documento). El 30-jul la decisión se revirtió. Los números UC13 y UC14 ya se habían reutilizado para la cartilla de fidelidad, así que este rol usa **UC18–UC20**.

Es un administrador **del lado de Aliflow**, no de ningún local. Es el único rol con visibilidad sobre todos los tenants.

UC18 – Dar de alta un local y crear su vista de proveedor

Campo	Detalle
Actor primario	Super-Admin de Aliflow
Precondición	Existe un acuerdo comercial con el local.
Flujo principal	1. El Super-Admin registra el local nuevo con sus datos (nombre comercial, RUC, puntos de entrega).2. Crea su vista de proveedor y la primera cuenta de Proveedor del local.3. Configura la integración con el ERP de ese local (<<include>> UC7).4. El local queda habilitado para publicar menú y vender.
Postcondición	El local opera como un tenant más de la plataforma.
Qué cierra	El punto que quedó abierto el 28-jul: <i>“si ningún rol del sistema da de alta un local nuevo, ese paso es manual y fuera de alcance”</i> . Ya no es manual ni está fuera de alcance.

UC19 – Brindar soporte a los locales

Campo	Detalle
Actor primario	Super-Admin de Aliflow
Flujo principal	1. El Super-Admin consulta el estado de un local: órdenes, sincronización con su ERP, cupo reservado, incidencias.2. Diagnostica y resuelve el problema, o escala al equipo técnico.
Regla de alcance	Es el único rol que puede ver información de más de un local. Cada acción queda en RegistroAuditoria.

UC20 – Administrar la plataforma

Campo	Detalle
Actor primario	Super-Admin de Aliflow

Campo	Detalle
Flujo principal	Configuración general de Aliflow: activar o desactivar locales, parámetros globales y monitoreo del estado de las integraciones de todos los tenants.

Casos de uso incluidos/extendidos (sub-flujos reutilizables)

Estos no son procesos de negocio independientes, sino pasos reutilizados dentro de los casos de uso principales (relación <<include>>/<<extend>> en el diagrama). Se documentan de forma breve:

ID	Nombre	Incluido/extiende en	Propósito
UC1a	Autenticar con proveedor de identidad	UC1 (include)	Delegar la autenticación en Google OAuth 2.0 / OIDC.
UC1b	Crear perfil y tarjeta virtual	UC1 (extend, solo primer ingreso)	Inicializar el perfil del estudiante con saldo en cero.
UC2a	Generar comprobante de recarga	UC2 (include)	Emitir constancia sin validez tributaria de la recarga.
UC3a	Sincronizar catálogo con sistema del proveedor	UC3 (include)	Obtener platos/precio/stock actualizados vía la API genérica.
UC4a	Validar saldo y disponibilidad	UC4 (include)	Revalidar saldo suficiente y stock justo antes de confirmar la compra.
UC4b	Generar comprobante de compra	UC4 (include)	Emitir constancia interna sin validez tributaria (ver UC11).
UC4c	Notificar venta al sistema del proveedor	UC4 (include)	Descontar stock en el ERP externo vía la API genérica (patrón outbox si falla, ver arquitectura).
UC5a	Buscar orden (código o manual)	UC5 (include)	Ubicar la orden por código de validación o por nombre/ID institucional.
UC5b	Validar y confirmar entrega	UC5 (include)	Verificar estado, cambiar a “Entregado”, invalidar código, registrar timestamp/operador.
UC5c	Manejar excepción de entrega	UC5 (extend)	Código inválido/ya usado, o fallo de conectividad (riesgo v1 sin modo offline).
UC5d	Acreditar sello en la cartilla	UC5 (include)	Sumar un sello a la cartilla vigente del estudiante en ese local, si el local tiene programa de fidelidad activo y

ID	Nombre	Incluido/extiende en	Propósito
			no se acreditó ya un sello ese día.

Documento preparado por el Grupo de Ingeniería como parte del entregable 02.a (Diagramas de casos de uso y documentación completa de cada caso de uso).

Documentación del Diagrama de Clases — Aliflow

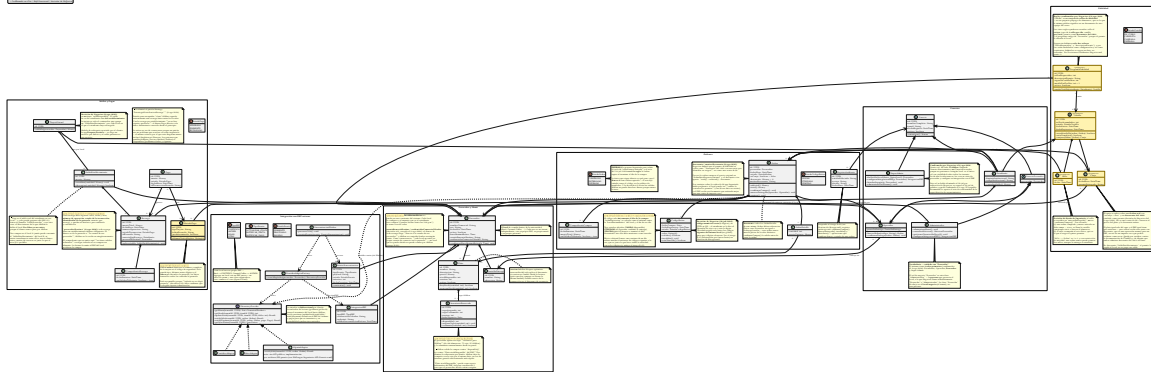


Figure 2: Diagrama de clases de Aliflow

Diagrama fuente: ../../uml/diagrama-clases.puml (mismo directorio) — ver ../../uml/README-diagramas.md para regenerar el SVG tras editarlo.

Este diagrama modela la **lógica de negocio** del sistema (no las clases de infraestructura/ORM/framework), organizada en 6 paquetes.

Convención visual (revisión 27-jul-2026): los elementos con fondo amarillo y estereotipo <<propuesta>> son diseño de Ingeniería sin validar todavía con Negocios — no decisiones ya confirmadas. Antes esta distinción solo vivía en este markdown; ahora es visible directamente en el .svg, para que no se pueda confundir una propuesta con una decisión cerrada solo mirando la imagen.

1. Paquete “Usuarios”

Actualizado el 8-ago-2026: los 4 roles quedaron confirmados por Negocios. Estudiante, Proveedor y Operador operan **dentro de un local**; el **Super-Admin** es de Aliflow y está por encima de todos. El rol “Administrador” sigue siendo el Proveedor — el gerente del local.

Sobre el vaivén, que conviene registrar: el 28-jul Negocios indicó que el super-admin **no existía** y se eliminó del diagrama. El 30-jul esa decisión se revirtió de palabra, y el 8-ago quedó confirmada formalmente. Es un caso concreto de lo que advierte el riesgo R-08: rehacer diagramas por una decisión que cambia dos días después tiene un costo real.

SuperAdmin extiende Usuario directamente, no UsuarioProveedor — y esa distinción es deliberada: UsuarioProveedor obliga a pertenecer a un local (# proveedor: Proveedor), y el Super-Admin es el único rol **sin** local, con visibilidad sobre todos los tenants. Da de alta locales nuevos, les crea su vista de proveedor, configura su integración con el ERP y brinda soporte.

Jerarquía de herencia con **Usuario** como clase abstracta base (id, nombreCompleto, email, fechaRegistro, activo), especializada en:

- **Estudiante** — mapea al actor “Estudiante”.
- **UsuarioProveedor** (abstracta) — una persona con acceso al panel de un local específico (# proveedor: Proveedor). Se especializa en:
 - **Administrador** — mapea al actor “**Proveedor**”: administra menú, métricas, la integración con el ERP del local, y las cuentas del personal de su propio local (UC12).
 - **Operador** — mapea al actor “Operador”: valida las compras de los estudiantes y marca la entrega física. Asociado a un PuntoDeEntrega específico.

Nota de vocabulario (importante para no perderse en el diagrama): la palabra “Proveedor” designa dos cosas distintas.

En el lenguaje de Negocios	En el diagrama de clases	Qué es
Rol “ Proveedor ” (= “Administrador”, el gerente)	clase Administrador	una persona con cuenta en Aliflow
El local / proveedor de alimentación (Barú, Caramel Coffee)	clase Proveedor	el negocio : el tenant, con su menú, su ERP y su personal

Se conservó Proveedor como nombre de la entidad-negocio porque así se usa en el acta (“proveedores de alimentación”) y en todo el resto del modelo (SaldoProveedor, IntegracionERP, tenantId). Renombrarla habría propagado churn a ~10 archivos sin ganar claridad real; la tabla de arriba y una nota dentro del propio diagrama resuelven la ambigüedad.

Qué se eliminó: la clase Administrador que colgaba directamente de Usuario (el super-admin, con darDeAltaProveedor() y gestionarUsuarios()), y las clases Propietario/Cajero, renombradas a Administrador/Operador para usar el vocabulario oficial de Negocios en vez del informal del equipo.

Personal múltiple por local — confirmado, ya no es propuesta. Negocios confirmó (28-jul-2026) que un local puede tener **varias** cuentas de Proveedor y varias de Operador. Las cardinalidades quedaron en Proveedor “1” *-- “1..*” Administrador (al menos un gerente) y Proveedor “1” *-- “0..*” Operador. Estas clases perdieron el estereotipo <<propuesta>> y el fondo amarillo.

2. Paquete “Proveedor y Menú”

Proveedor (el tenant/negocio — en la práctica, cada local de comida de la universidad: Barú, Caramel Coffee, etc.) agrega su personal (UsuarioProveedor), sus puntos de entrega físicos (PuntoDeEntrega) y su menú (Plato). Plato.hayStock() encapsula la validación de disponibilidad que en el flujo se menciona como “revalidación de stock justo antes de confirmar” (est-4).

Control de concurrencia (agregado 27-jul-2026): Plato ahora tiene un campo `version` y un método `reservarStock(cantidad)` — implementa bloqueo optimista para el riesgo ya identificado desde el flujo original (“dos estudiantes no deben poder comprar la última unidad simultáneamente”). Antes este riesgo estaba documentado en prosa pero no tenía ningún elemento correspondiente en el diagrama de clases; el detalle exacto de la transacción se completa en el diagrama de secuencia.

Validado empíricamente (27-jul-2026, ver ../.../demo-odoo/README.md sección 7): se probó implementar este mismo bloqueo directamente contra el ERP externo (Odoo, vía RPC) y falló bajo concurrencia real (5 hilos comprando con 3 unidades de stock vendieron las 5). Esto confirma que `Plato.version/reservarStock()` deben vivir en la base de datos propia de Aliflow, no delegarse al ERP — la prueba de concurrencia repetida contra un dominio local con lock real (`../.../demo-odoo/plato_local.py`) sí se comportó correctamente.

InventarioReservado — agregado el 30-jul-2026

La clase nueva más importante de esta revisión, y la que cambia cómo se decide si Aliflow puede vender.

El problema que resuelve. El ERP del local descuenta al instante cuando alguien compra en caja, pero Aliflow puede tardar en enterarse. En esa ventana, un estudiante compra algo que ya no existe. Son **dos escritores sobre el mismo contador** sin transacción común: sincronizar más seguido achica la ventana, no la cierra.

Cómo lo resuelve. El proveedor aparta un cupo exclusivo para Aliflow (de 100 almuerzos: 75 a caja, 25 a Aliflow) y lo administra desde su panel (UC16). **Aliflow valida la compra contra InventarioReservado.disponible(), no contra Plato.stockDisponible.**

Por qué es la solución correcta y no un parche: convierte un problema de consistencia distribuida —difícil, y sin solución completa cuando no se controlan los dos sistemas— en uno de **partición de recursos**, que es trivial. Aliflow deja de competir por un dato compartido porque pasa a ser dueño exclusivo del suyo.

`Plato.stockDisponible` no desaparece: queda como **espejo informativo** del ERP, útil para conciliar y para que el proveedor decida cuánto asignar. `InventarioReservado` conserva su propio `version` para el bloqueo optimista, porque dos estudiantes sí pueden pelear por la última unidad **del cupo**.

Costo que introduce, registrado como riesgo R-20: el cupo depende de que el proveedor lo mantenga al día. Si no lo repone, Aliflow muestra “agotado” mientras el local tiene comida — y el fallo es silencioso, porque nadie reclama por lo que no puede comprar.

3. Paquete “Wallet y Pagos”

Reescrito el 8-ago-2026 con la decisión #13 de Negocios: la recarga es por establecimiento. Es el cambio más grande que sufrió este paquete, y revierte la decisión #4 del 28-jul.

La regla nueva, en una línea: el saldo pertenece al establecimiento, no al estudiante. El estudiante recarga *para un local* y solo puede gastar ahí. El dinero va de la pasarela **directo a la cuenta de ese proveedor**; Aliflow no lo recibe ni lo custodia en ningún momento.

El modelo de referencia lo aportó el propio cliente: la aplicación **Parqueo Positivo**, donde el usuario debe elegir un servicio por defecto antes de operar, un indicador permanente le recuerda en cuál está, y el saldo que ve pertenece a ese servicio.

Qué cambió en las clases

Clase	Antes (saldo único)	Ahora (por establecimiento)
TarjetaVirtual	Tenía <code>saldoDisponible</code> : una bolsa única gastable en cualquier local	Pierde <code>saldoDisponible</code>. Queda como contenedor que agrupa un <code>SaldoEstablecimiento</code> por cada local donde el estudiante recargó. Expone <code>saldoEn(proveedor)</code>
<code>SaldoProveedor</code> → <code>SaldoEstablecimiento</code>	Era el libro interno de lo que Aliflow le debía a cada local, acreditado al comprar	Es el saldo real del estudiante en ese local. El libro de deuda desapareció : Aliflow no le debe nada a nadie porque el dinero nunca pasó por sus manos
Recarga	Se aplicaba sobre la bolsa única	Gana <code>proveedorDestino</code> obligatorio. Toda recarga tiene un establecimiento destino
Proveedor	—	Gana <code>credencialesComercioCifradas</code> : cada local necesita su propia cuenta de comercio en la pasarela
<code>EstrategiaDistribucionRecarga</code> + <code>DistribucionBajoDemanda</code>	Patrón Strategy que encapsulaba cómo se repartía la recarga entre locales	Eliminados. Ver abajo

Por qué se eliminó el patrón Strategy, y no se conservó “por las dudas”

`EstrategiaDistribucionRecarga` existía para encapsular *cómo* Aliflow repartía internamente una recarga única entre los locales. **Con la recarga por establecimiento no hay reparto que hacer**: el dinero tiene un único destinatario, conocido desde antes de cobrar.

Se retira en vez de conservarse porque un patrón sin un problema que resolver es exactamente la sobre-ingeniería que este diagrama declara evitar — el mismo criterio por el que nunca incluyó Singleton ni Observer. Los patrones que quedan (Adapter, Factory Method, Outbox) responden a problemas reales y vigentes.

Vale registrar la ironía: el 28-jul se eliminó `RecargaDirectaPorProveedor` por considerarla descartada y se conservó `DistribucionBajoDemanda`. **Se eliminó la que resultó ser correcta.** Es un caso concreto del costo que advierte el riesgo R-08.

Lo que esta decisión resolvió

- **Desapareció la contradicción con el acta.** El §3.9 decía que el dinero llega directo a cada proveedor y que Aliflow no custodia fondos; ahora el modelo lo cumple literalmente.
- **Se desbloqueó el esquema de base de datos.** Este paquete era lo único congelado.

- **Se cerraron los riesgos R-18 y R-19.** El *split payments* dejó de ser necesario, porque cada recarga tiene un único destinatario.

Lo que esta decisión costó

- **Saldo fragmentado (riesgo R-21).** Un estudiante con \$6 en un local y \$4 en otro no puede comprar un almuerzo de \$7 en ninguno, teniendo \$10. Es tolerable porque el estudiante *elige* dónde pone su dinero —no es un reparto automático como la lectura A que se descartó el 28-jul—, pero va a ocurrir. Las mitigaciones son de interfaz y están en los requerimientos RF-07, RF-08, RF-12 y RF-15.
- **Cada local necesita su cuenta de comercio (riesgo R-22).** Un local que no pueda o no quiera abrirla no puede vender por Aliflow, aunque su ERP esté integrado.
- **Saldo huérfano (riesgo R-23).** Si el estudiante se gradúa o el local sale de la plataforma, queda saldo que **Aliflow no puede devolver porque nunca tuvo el dinero**. Se resuelve por contrato con cada local, no por software.

Lo que no cambió

Pago (con EstadoPago: APROBADO/PENDIENTE/RECHAZADO) genera una Recarga solo si es aprobado. ComprobanteRecarga sigue siendo el comprobante interno sin validez tributaria. MetodoPago sigue en amarillo: Aliflow nunca almacena el número completo de la tarjeta ni el código de seguridad, solo tipo, últimos cuatro dígitos y token — y falta confirmar que la pasarela elegida soporte tokenización, porque **todavía no se eligió pasarela** (decisión #12).

4. Paquete “Órdenes”

Orden agrega una o más OrdenDetalle (plato + cantidad + precio unitario — generalización razonable sobre el flujo, que describe la compra de un plato a la vez, pero sin costo de diseño adicional soporta más de un ítem). Tiene un CódigoRetiro propio (value object: valor, fechaExpiracion, usado).

Formato del código, cerrado el 28-jul-2026: Negocios eligió **código numérico corto de 6 dígitos**, descartando la propuesta previa de Ingeniería (UUID firmado con expiración). La razón es operativa y es buena: el estudiante se lo dice de viva voz al Operador, que lo digita en Aliflow para marcar el retiro — un UUID de 36 caracteres es impracticable para eso. Dos consecuencias de diseño quedaron anotadas en el diagrama:

1. **La unicidad se acota.** Seis dígitos son $\sim 10^6$ combinaciones: suficientes solo si la unicidad se exige entre los códigos **vigentes de un mismo local**, no globalmente ni de forma histórica. La generación reintenta ante colisión.
2. **El código pasa a ser adivinable.** Un UUID firmado no se puede adivinar; 6 dígitos sí. Registrado como riesgo **R-15** en ../01-Especificacion-de-Requerimientos/06-Gestion-de-Riesgos.md con su mitigación (límite de intentos + el hecho de que el Operador ve físicamente al estudiante).

EstadoOrden incluye **EXPIRADO** además de COMPRADO/ENTREGADO — esto no estaba en el flujo original; se agrega para cubrir el vacío ya detectado de “no hay estado para una orden nunca retirada” (../.../Hallazgos-Ingenieria-API-Generica.md, sección 5.3). Es una propuesta

de Ingeniería, marcada ahora también dentro del propio diagrama (nota amarilla junto a EstadoOrden), pendiente de que Negocios defina la regla exacta (después de cuánto tiempo expira, si hay reembolso, etc. — esto último sigue fuera de alcance de v1 según el acta).

ComprobanteCompra es el comprobante interno sin validez tributaria (est-4/prov-6), distinto de la factura real que el proveedor emite en su propio ERP.

Regla de un solo local por orden (corregida 27-jul-2026): Orden ahora tiene una asociación directa a Proveedor (no solo indirecta vía OrdenDetalle → Plato → Proveedor), con un invariante explícito en el diagrama: todos los OrdenDetalle de una misma Orden deben pertenecer a platos del mismo proveedor. Antes de esta corrección, el modelo permitía —sin querer— una orden con platos de proveedores distintos, lo cual habría roto el resto de la arquitectura (saldo por proveedor, un solo tenantId por llamada a notifySale, un evento de sincronización por orden). confirmarCompra() es responsable de validar este invariante antes de crear la orden.

Auditoría (agregada 27-jul-2026): RegistroAuditoria responde al riesgo R-09 (../01-Especificacion-de-Requerimientos/06-Gestion-de-Riesgos.md), que pedía explícitamente registro de auditoría para compras y redenciones — antes este requisito estaba documentado como riesgo pero no tenía ninguna clase correspondiente. Registra quién ejecutó confirmarCompra(), marcarEntregado(), invalidar() y la acreditación interna al local.

5. Paquete “Fidelidad” (requisito nuevo, 28-jul-2026)

Negocios pidió una **cartilla de fidelidad**: el estudiante acumula un sello por compra y al completar la cartilla gana un premio. **Las cinco reglas quedaron confirmadas el 8-ago-2026** y el paquete perdió el <<propuesta>>. Siguen sin definirse solo dos valores —cuántos sellos y cuál es el premio—, que por diseño son configuración y no constantes.

La decisión de diseño que evita quedarse esperando: los dos datos que faltan (sellosRequeridos, descripcionPremio) se modelan como **campos configurables de ProgramaFidelidad**, no como constantes. Cuando Negocios los defina, es un valor en base de datos — no hay que rediseñar ni reprogramar nada. Lo mismo con vigenciaCartillaDias (si Negocios decide que la cartilla no caduca, el campo queda nulo) y con maxSellosPorDia.

Cuatro decisiones de diseño que Ingeniería tomó y que Negocios confirmó el 8-ago-2026:

1. **El programa es por local, no de la plataforma.** ProgramaFidelidad cuelga de Proveedor. La razón es económica, no técnica: el premio lo regala el local, así que es el local quien debe poder decidir si lo ofrece, cuántos sellos pide y qué da. Un local puede no tener programa. Como consecuencia, el estudiante tiene **una cartilla activa por local**, no una sola global.
2. **El sello se acredita al entregar, no al comprar.** Sello se crea dentro de Orden.marcarEntregado(), no de confirmarCompra(). Si se acreditara al comprar, un estudiante podría llenar la cartilla comprando almuerzos y nunca yendo a buscarlos — el local pagaría el premio sin haber vendido nada real. Además el sello así acompaña al acto físico, que es lo que el negocio quiere premiar.
3. **Un sello por orden, garantizado por el modelo.** La asociación Sello --> Orden es 1 a 1 con restricción de unicidad. Si la confirmación de entrega se reintenta (fallo de red, doble clic

del Operador), la segunda inserción falla y no se acredita dos veces. Es el mismo principio de idempotencia que ya se usa en el outbox con eventId.

4. **Un sello por día como tope por defecto.** `maxSellosPorDia = 1`. Sin este límite, la cartilla premia volumen en vez de recurrencia, y se puede llenar en un solo día comprando el ítem más barato del menú varias veces. Este punto depende de lo que Negocios haya querido decir con “10 veces diarias” — ver `../.../Decisiones-Pendientes-Negocios.md`, punto 9.

El canje toca el resto del sistema en tres lugares:

- **Orden.esCanje + descuento + motivoDescuento** — el canje se aplica sobre una orden real con **descuento del 100% rotulado como premio**, no con total \$0. Tiene que ser una orden de verdad porque el plato igual sale del cupo y el estudiante igual necesita un código de retiro. *(Ingeniería había propuesto la orden de \$0; Negocios pidió el descuento el 8-ago-2026, y es mejor: conserva el precio original, así el local puede ver cuánto le costaron los premios — un dato que con \$0 no existía.)*
- **La wallet no se toca.** No se descuenta el SaldoEstablecimiento del estudiante: el premio lo regala el local, no se paga con saldo.
- **El ERP sí se entera, y el problema que había aquí se resolvió.** Antes la duda era si un `notifySale` de \$0 sería rechazado por el ERP como error, y si había que emitirlo como documento de cortesía o como descuento. **La respuesta de Negocios lo decidió: descuento del 100%**, que además es una operación normal para cualquier ERP. Queda solo verificar contra la documentación de Contífico y de Alpwin que ambos lo admiten en línea de venta.

Concurrencia: el paso `COMPLETA` → `CANJEADA` usa el mismo mecanismo atómico y condicional que la redención del código de retiro (`UPDATE ... WHERE estado = 'COMPLETA'`). Si dos pestañas del estudiante intentan canjear a la vez, la segunda afecta 0 filas y falla sin crear la orden.

6. Paquete “Integración con ERP externo”

Materializa directamente la arquitectura ya diseñada en `../.../Hallazgos-Ingenieria-API-Generica.md` (sección 3):

- **IInventoryProvider** (interfaz/puerto) — el core de Aliflow (representado aquí por la dependencia `Orden ..> IInventoryProvider`) solo conoce esta abstracción, nunca un ERP concreto.
- **ContificoAdapter, AlpwinAdapter, OdoAdapter** — adaptadores concretos (patrón **Adapter**). `AlpwinAdapter` lleva una nota explícita: al no tener API pública, su implementación real sería por archivos/BD puente, no llamadas síncronas.
- **ProviderAdapterFactory** — patrón **Factory Method**: dado un `Proveedor`, lee su `IntegracionERP.tipoERP` y devuelve la implementación concreta correspondiente. Nadie más en el sistema necesita un `switch/if` sobre el tipo de ERP.

Actualizado el 28-jul-2026 — este paquete dejó de ser una previsión y pasó a ser un requisito duro. Negocios aclaró que Aliflow debe poder conectarse al ERP de **cualquier** local de la universidad, y que cada uno tiene el suyo: Barú usa **Contífico**, Caramel Coffee usa **Alpwin**, y así sucesivamente. Tres consecuencias sobre el modelo:

1. **Varios adaptadores conviven en producción**, no uno. Antes se asumía un único ERP objetivo y la factory era casi decorativa; ahora es la pieza que hace funcionar el multi-tenant real. El diseño no cambió — es exactamente lo que el patrón Adapter + Factory estaba pensado para soportar — pero pasó de “buena práctica defensiva” a “sin esto el producto no existe”.
 2. **La interfaz es bidireccional** y eso ahora está explícito. getMenu()/getStock() traen el inventario del local hacia Aliflow; updateStock()/notifySale() y el nuevo **notifyPayment()** devuelven al ERP las órdenes y los pagos, para que su inventario y su contabilidad queden sincronizados. Se agregó TipoEvento.NOTIFICAR_PAGO al outbox por la misma razón.
 3. **TipoERP cambió de contenido y de rol**. Ahora es CONTIFICO | ALPWIN | OD00 | OTRO, con Contífico primero por ser el del local piloto. El valor OTRO es deliberado: agregar un local con un ERP desconocido debe ser un valor de enum más una clase adaptadora, sin tocar el core.
- **EventoSincronizacion + SincronizacionWorker** — implementan el patrón **Outbox** ya diseñado: cada venta genera un evento (TipoEvento.NOTIFICAR_VENTA), que el worker procesa con reintentos (EstadoEvento: PENDIENTE/PROCESADO/FALLIDO), evitando la “venta huérfana” ya identificada como riesgo (R-02 en ../01-Especificacion-de-Requerimientos/06-Gestion-de-Riesgos.md).

Principios SOLID aplicados

Principio	Dónde se aplica
S — Responsabilidad única	Orden gestiona su propio estado y ciclo de vida; la traducción a cada ERP vive exclusivamente en su adaptador; SincronizacionWorker solo orquesta reintentos, no lógica de negocio de la venta.
O — Abierto/cerrado	Agregar un ERP nuevo = una clase NuevoErpAdapter nueva, cero cambios al core (IInventoryProvider ya definido). <i>(El otro ejemplo que había aquí, EstrategiaDistribucionRecarga, se eliminó el 8-ago-2026 al quedar sin problema que resolver — ver sección 3.)</i>
L — Sustitución de Liskov	Cualquier IInventoryProvider (Contífico/Alpwin/Odoo) es intercambiable sin que el código que lo usa (SincronizacionWorker, Orden) se entere de la diferencia. Con varios locales activos a la vez, esto se ejercita de verdad en producción, no solo en teoría.
I — Segregación de interfaces	IInventoryProvider se mantiene deliberadamente pequeña (6 métodos, todos relacionados a inventario/venta/pago) — no se mezcla con responsabilidades de facturación fiscal, que quedan fuera de esta interfaz.
D — Inversión de dependencias	ProviderAdapterFactory y SincronizacionWorker dependen de la abstracción IInventoryProvider, nunca de OdooAdapter/ContificoAdapter/AlpwinAdapter directamente.

Patrones de diseño usados (y por qué, no solo “porque sí”)

- **Adapter** — resuelve directamente el reto técnico central del proyecto (integrar con ERPs heterogéneos sin acoplarse a ninguno).
- **Factory Method** (ProviderAdapterFactory) — evita que la lógica de “qué adaptador usar” se disperse por el código; centraliza la decisión en un solo lugar.
- **Strategy** (~~EstrategiaDistribucionRecarga~~) — **eliminado el 8-ago-2026**. Servía para encapsular cómo se repartía internamente una recarga única entre locales. Con la recarga por establecimiento ya no hay reparto, así que el patrón se quedó sin problema que resolver y se retiró en vez de conservarse por las dudas. Ver sección 3.
- **Outbox** (arquitectural, no GoF clásico) — ya validado técnicamente en el demo con Odoo Community (../.../Hallazgos-Ingenieria-API-Generica.md, sección 4.2).

No se forzaron patrones adicionales (ej. Singleton, Observer) donde no había un problema real que resolver — evitar ese “mal olor” de sobre-ingeniería fue una decisión deliberada. **Y el criterio se aplicó también en sentido inverso:** cuando la decisión #13 dejó al Strategy sin propósito, se lo eliminó. Conservar un patrón que ya no resuelve nada habría sido el mismo mal olor, solo que heredado.

Malos olores evitados

- **God class:** no existe una clase “Sistema” o “AliflowService” que concentre toda la lógica — cada responsabilidad vive en la clase del dominio que le corresponde.
- **Primitive obsession:** CódigoRetiro y los comprobantes son objetos propios (con su propia validación) en vez de campos sueltos tipo String codigados por otras clases.
- **Acoplamiento a implementación externa:** ninguna clase de dominio (Orden, Plato, Proveedor) importa o conoce un SDK específico de Odoo/Contífico — todo pasa por IInventoryProvider.

Supuestos y pendientes de este diagrama (a validar con el equipo/Negocios)

Todos marcados con <<propuesta>> y fondo amarillo directamente en el diagrama. Tras las respuestas de Negocios del 8-ago-2026, **la lista bajó de 4 supuestos a 1:**

1. Usuario.autenticar() no distingue aún el mecanismo (OAuth institucional para Estudiante vs. credenciales propias para los demás roles) a nivel de firma — se resuelve en ../.../uml/actividad-autenticacion.puml.

Cerrados el 8-ago-2026:

- **SaldoProveedor como libro interno** — el supuesto desapareció junto con la clase: con recarga por establecimiento no hay libro de deuda que llevar. Ver sección 3.
- **EstadoOrden.EXPIRADO** — el acta del 30-jul cerró la regla (la orden expira al terminar el día de la compra). Lo único que sigue abierto es qué pasa con el **dinero** de una orden expirada, que es una decisión de negocio, no un supuesto de modelado.

- **Todo el paquete “Fidelidad”** — las cinco reglas quedaron confirmadas: es tarjeta de sellos y no paquete prepago, sello al retirar, tope de 1/día, cartilla por local, y premio como descuento del 100%. El paquete perdió el <<propuesta>>.

Cerrados por Negocios el 28-jul-2026: la jerarquía de personal por local, el alcance del rol Administrador (que resultó ser el Proveedor mismo) y el formato del código de retiro.

Sigue en amarillo MetodoPago, pero no por un supuesto de modelado: depende de que se elija pasarela y de confirmar que soporte tokenización (decisión #12).

Cambios aplicados en la revisión del 28-jul-2026 (decisiones de Negocios)

Negocios resolvió cinco de las decisiones que bloqueaban el modelo. Impacto sobre este diagrama:

#	Decisión de Negocios	Cambio en el diagrama
1	Aliflow se conecta al ERP de cualquier local; cada uno usa el suyo (Barú → Contífico, Caramel Coffee → Alpwin) y la conexión es bidireccional	TipoERP reordenado y con OTRO; notifyPayment() agregado a IInventoryProvider; TipoEvento.NOTIFICAR_PAGO agregado al outbox
2	Solo 3 roles; “Administrador” = “Proveedor” = gerente del local	Eliminada la clase Administrador de plataforma; Propietario→Administrador, Cajero→Operador; marcarEntregado(operador)
3	Un local puede tener varios Proveedores y varios Operadores	Cardinalidades 1..* / 0..*; la jerarquía UsuarioProveedor deja de ser <<propuesta>>
4	Recarga única distribuida internamente	TarjetaVirtual.saldoDisponible agregado; SaldoProveedor pasa a libro interno (montoAcumulado); RecargaDirectaPorProveedor eliminada, DistribucionBajoDemanda vigente
6	Código de retiro numérico corto	CodigoRetiro.valor documentado como 6 dígitos; nota de unicidad acotada por local; riesgo R-15
9	Cartilla de fidelidad (requisito nuevo)	Paquete “Fidelidad” completo: ProgramaFidelidad, Cartilla, Sello, Canje, EstadoCartilla; campo Orden.esCanje; riesgo R-17

Lo que estas decisiones mejoraron y lo que complicaron, dicho sin adornos:

- **Mejor:** el local piloto pasó de ser el caso imposible (Alpwin, sin API) al caso fácil (Contífico, API REST documentada). El riesgo que más pesaba en el proyecto (R-11) dejó de bloquear el arranque.
- **Más difícil:** el sistema ya no es “Aliflow + un proveedor”, sino una plataforma multi-tenant con N ERP heterogéneos conviviendo. Eso encarece pruebas, credenciales, monitoreo y soporte — y hace que la capa de integración deje de ser opcional.

- **Sin resolver:** cuándo ocurre exactamente la distribución interna del saldo (ver sección 3), y quién da de alta un local nuevo ahora que no existe un super-admin (ver ../02-Modelamiento-Parte-Estatica/a-Casos-de-Uso.md, UC12).

Correcciones aplicadas en esta revisión (27-jul-2026)

A partir de una autoevaluación crítica del diseño hasta este punto, se corrigieron 3 inconsistencias reales y se cerraron 2 vacíos:

1. **Inconsistencia — orden multi-proveedor no prevenida:** corregida con la asociación directa Orden → Proveedor + invariante explícito (ver sección 4 arriba).
2. **Inconsistencia — sin distinción visual entre confirmado y propuesto:** corregida con la convención <<propuesta>> + leyenda de colores, aplicada en este diagrama y en ../../uml/casos-de-uso.puml/../../uml/diagrama-componentes.puml.
3. **Vacío — sin control de concurrencia modelado:** corregido con Plato.version + reservarStock() (bloqueo optimista).
4. **Vacío — sin clase de auditoría pese a que R-09 la pedía explícitamente:** corregido con RegistroAuditoria.
5. **Riesgo R-01 desactualizado** tras el hallazgo de Alpwin: corregido en ../01-Especificacion-de-Requerimientos/06-Gestion-de-Riesgos.md, no en este diagrama directamente.

Documentación de Diagramas de Objetos — Aliflow

Dos instantáneas (snapshots) de instancias concretas, cada una ilustrando un aspecto medular del diseño que el diagrama de clases (../../uml/diagrama-clases.puml) por sí solo no deja tan claro.

1. Billetera y orden de un estudiante

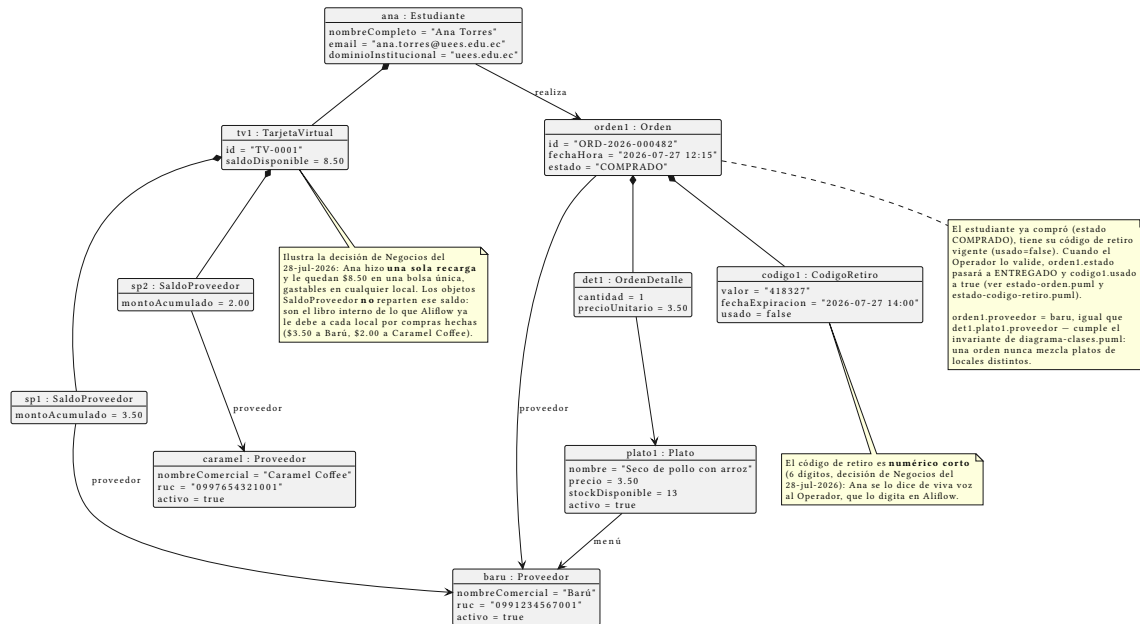


Figure 3: Diagrama de objetos: billetera y orden

Fuente: ../../uml/objeto-billetera-orden.puml

Qué ilustra: cómo funciona en concreto la decisión de Negocios del 28-jul-2026 sobre la recarga, que en el diagrama de clases solo se ve como una cardinalidad 1 -- 0..*. Ana Torres (Estudiante) tiene una única TarjetaVirtual con **\$8.50 en una bolsa común**, gastables en cualquier local — hizo una sola recarga, no una por proveedor. Los dos objetos SaldoProveedor colgando de esa tarjeta **no reparten** ese saldo: son el libro interno de lo que Aliflow ya le debe a cada local por compras hechas (\$3.50 a Barú, \$2.00 a Caramel Coffee). Esa es la lectura que Ingeniería le dio a “Aliflow distribuye internamente a todos los proveedores”, y está marcada como interpretación pendiente de confirmar en ../.././Decisiones-Pendientes-Negocios.md.

La orden (ORD-2026-000482) está en estado COMPRADO, con su CodigoRetiro todavía sin usar — la instantánea representa el momento justo después de la compra y antes del retiro físico del almuerzo. Ese código (418327) refleja el otro cambio del 28-jul: **numérico corto de 6 dígitos**, no UUID.

2. Integración multi-tenant con los ERP de dos locales reales (Barú/Contífico y Caramel Coffee/Alpwin)

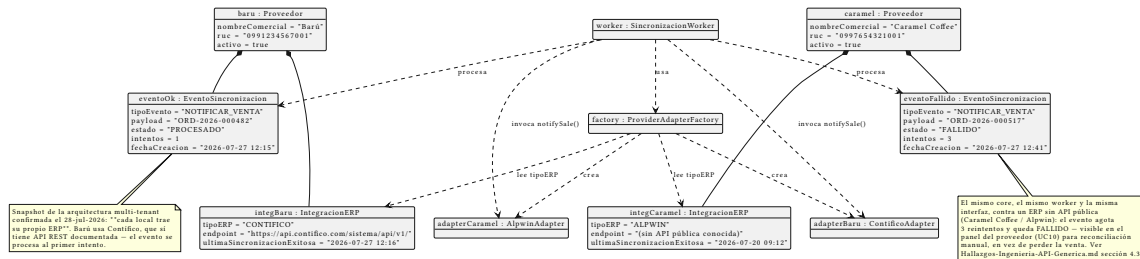


Figure 4: Diagrama de objetos: integración ERP

Fuente: `../../../../uml/objeto-integracion-erp.puml`

Qué ilustra: la razón de ser de toda la capa de integración, ahora con los dos locales reales confirmados por Negocios el 28-jul-2026 conviviendo en el mismo snapshot:

- **Barú** tiene su IntegracionERP con tipoERP = CONTIFICO, un SaaS con API REST documentada. Su evento de venta se procesa **al primer intento** (PROCESADO).
- **Caramel Coffee** tiene tipoERP = ALPWIN, un sistema sin API pública conocida. Su evento **agota 3 reintentos y queda FALLIDO**.

La misma ProviderAdapterFactory, el mismo SincronizacionWorker y el mismo contrato IInventoryProvider sirven a los dos casos — que es exactamente lo que el patrón Adapter debía demostrar. Antes de la corrección de Negocios, este diagrama asumía que Barú usaba Alpwin y mostraba un único caso fallido; el escenario real es mejor de lo que creíamos (el local piloto es el fácil) y a la vez más exigente (hay que sostener varios ERP a la vez, no uno).

El evento fallido no es un caso de error hipotético: es exactamente el escenario que el patrón Outbox (`../../../../Hallazgos-Ingenieria-API-Generica.md` sección 3.4) fue diseñado para manejar sin perder la venta — queda visible para reconciliación manual (caso de uso UC10, `../02-Modelamiento-Parte-Estatica/a-Casos-de-Usos.md`) en vez de desaparecer silenciosamente.

Ambos diagramas usan datos de ejemplo — no corresponden a transacciones reales.

Documentación del Diagrama de Componentes — Aliflow

proyecto de Ingeniería, pendiente de validar con Negocio
confirmado en uso / Bajo funcional / decisión de Negocio

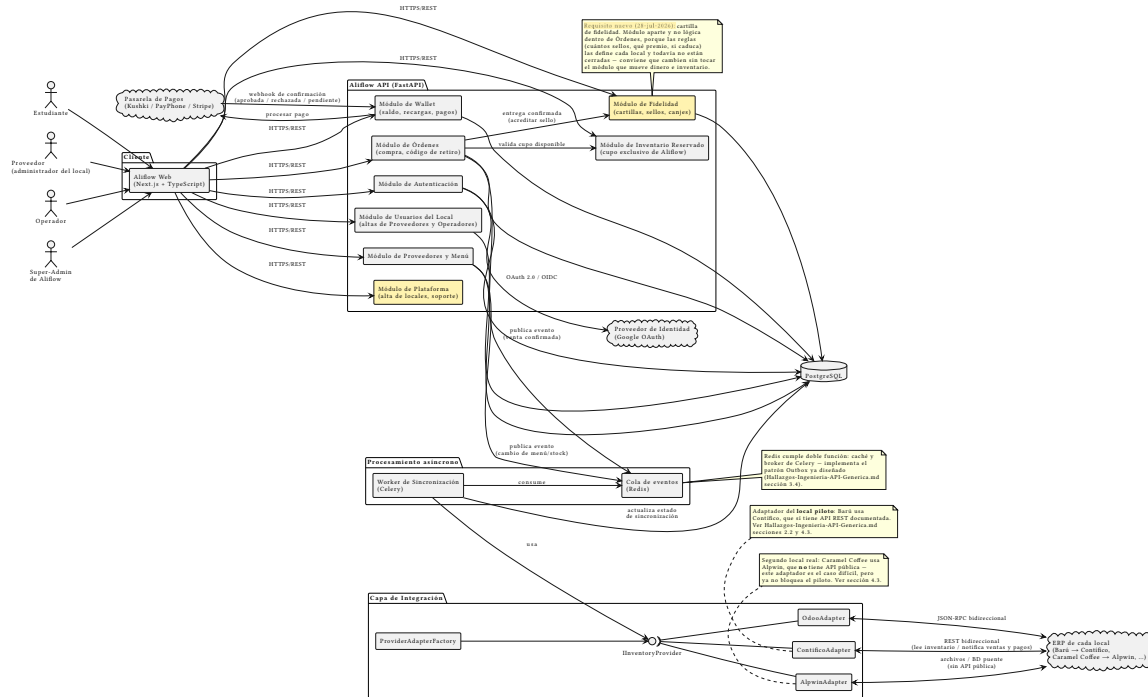


Figure 5: Diagrama de componentes de Aliflow

Diagrama fuente: ../../uml/diagrama-componentes.puml.

Este diagrama incorpora, por primera vez en la documentación formal del proyecto, el **stack tecnológico** recuperado de investigación previa del equipo (conversación revisada el 26-jul-2026): Next.js en el cliente, FastAPI en el backend, PostgreSQL como base de datos, y Redis/Celery para procesamiento asincrónico. Hasta ahora esta información solo existía en una conversación externa — queda aquí formalizada como parte del diseño.

Convención visual (revisión 27-jul-2026): el “Módulo de Administración” aparece con fondo amarillo — misma convención que en ../../uml/diagrama-clases.puml y ../../uml/casos-de-uso.puml — porque expone los casos de uso UC12-UC14, propuesta de Ingeniería sin validar con Negocio.

Componentes principales

Cliente

- **Aliflow Web (Next.js + TypeScript)** — única aplicación web que sirve a los **3 roles** del sistema (Estudiante, Proveedor, Operador — decisión de Negocio del 28-jul-2026; no existe un rol de super-admin de plataforma), consumiendo la API vía HTTPS/REST. No hay apps nativas separadas por rol; la diferenciación de interfaz ocurre por rol de sesión, no por despliegue distinto.

Aliflow API (FastAPI)

Organizada en módulos, cada uno mapeado a los paquetes del diagrama de clases: - **Módulo de Autenticación** — implementa UC1 (OAuth institucional) y UC6 (login operativo); es el único módulo que habla con el **Proveedor de Identidad** externo. - **Módulo de Wallet** — implementa UC2 (recarga); es el único módulo que habla con la **Pasarela de Pagos**. - **Módulo de Órdenes** — implementa UC4/UC5 (compra, retiro); publica eventos a la cola cuando se confirma una venta. - **Módulo de Proveedores y Menú** — implementa UC7/UC8 (integración, administración de menú). - **Módulo de Fidelidad** (*nuevo, 28-jul-2026*) — implementa UC13/UC14/UC15 y el sub-flujo UC5d: cartillas, sellos y canjes. Se modela como módulo aparte y no como lógica dentro de Órdenes por una razón concreta: las reglas del programa (cuántos sellos, qué premio, tope diario, si caduca) las define cada local y todavía no están cerradas. Conviene que puedan cambiar sin tocar el módulo que mueve dinero e inventario. La flecha Órdenes → Fidelidad refleja que el sello se acredita cuando la entrega se confirma. - **Módulo de Usuarios del Local** — implementa UC12: permite que un Proveedor (administrador del local) dé de alta y revoque a otros Proveedores y a los Operadores de **su propio** local. Reemplaza al antiguo “Módulo de Administración”, que existía para un rol de super-admin de plataforma ya descartado por Negocios (28-jul-2026).

Todos los módulos persisten en **PostgreSQL** directamente (no hay una capa de repositorio separada mostrada aquí — a ese nivel de detalle correspondería un diagrama de clases de infraestructura, fuera del alcance de “lógica de negocio” que pide la rúbrica).

Capa de Integración

Es la traducción directa a componentes del patrón Adapter ya diseñado (`../../Hallazgos-Ingenieria-API-Generica.md` sección 3, `../../uml/diagrama-clases.puml`): la interfaz **InventoryProvider** (notación de lollipop) es implementada por `ContificoAdapter`, `AlpwinAdapter` y `OdoAdapter`, y `ProviderAdapterFactory` decide cuál instanciar **según el local**. Ningún otro componente del sistema depende de un adaptador concreto — solo de la interfaz.

Esta capa dejó de ser una previsión y pasó a ser un requisito duro con la confirmación de Negocios del 28-jul-2026: **cada local de la universidad usa un ERP distinto** (Barú → Contífico, Caramel Coffee → Alpwin, etc.), así que en producción van a convivir varios adaptadores simultáneamente, no uno solo. Las flechas hacia el ERP se dibujan **bidireccionales**: Aliflow lee inventario y menú desde el ERP del local, y le devuelve las órdenes y los pagos para que su inventario quede sincronizado.

Procesamiento asíncrono

- **Cola de eventos (Redis)** — recibe eventos publicados por Órdenes y por Proveedores/Menú (venta confirmada, cambio de stock).
- **Worker de Sincronización (Celery)** — consume la cola, usa la Capa de Integración para notificar al ERP externo, y actualiza el estado de sincronización en la base de datos. Es la implementación concreta del patrón Outbox.

Sistemas externos

- **Proveedor de Identidad (Google OAuth)** — ya documentado desde el flujo original (est-1).

- **Pasarela de Pagos (Kushki / PayPhone / Stripe)** — nueva incorporación formal; son las opciones que se manejaron en la investigación previa del equipo para el riesgo R-06 (../01-Especificacion-de-Requerimientos/06-Gestion-de-Riesgos.md) sobre limitaciones del ambiente de pruebas de pagos.
- **ERP de cada local** — no es un solo sistema: Barú usa **Contífico** (API REST documentada, el caso fácil y el del local piloto) y Caramel Coffee usa **Alpwin** (sin API pública, integración por archivos/BD puente). Ver notas en el diagrama y ../ ../ ../Hallazgos-Ingenieria-API-Generica.md sección 4.3.

Decisiones de este diagrama que quedan abiertas

1. **Elección final de pasarela de pago** (Kushki vs. PayPhone vs. Stripe) — no se ha decidido, solo se documentan como candidatas evaluadas previamente.
2. **Redis con doble rol** (caché + broker de Celery) es una simplificación de infraestructura razonable para el alcance del proyecto universitario; en un entorno de producción más grande podría separarse en dos servicios.
3. El **Módulo de Administración** expone los casos de uso UC12-UC14 que Ingeniería propuso pero Negocios no ha validado — se incluye aquí por consistencia con el diagrama de clases, no porque su alcance esté cerrado.

Documentación del Diagrama de Despliegue — Aliflow

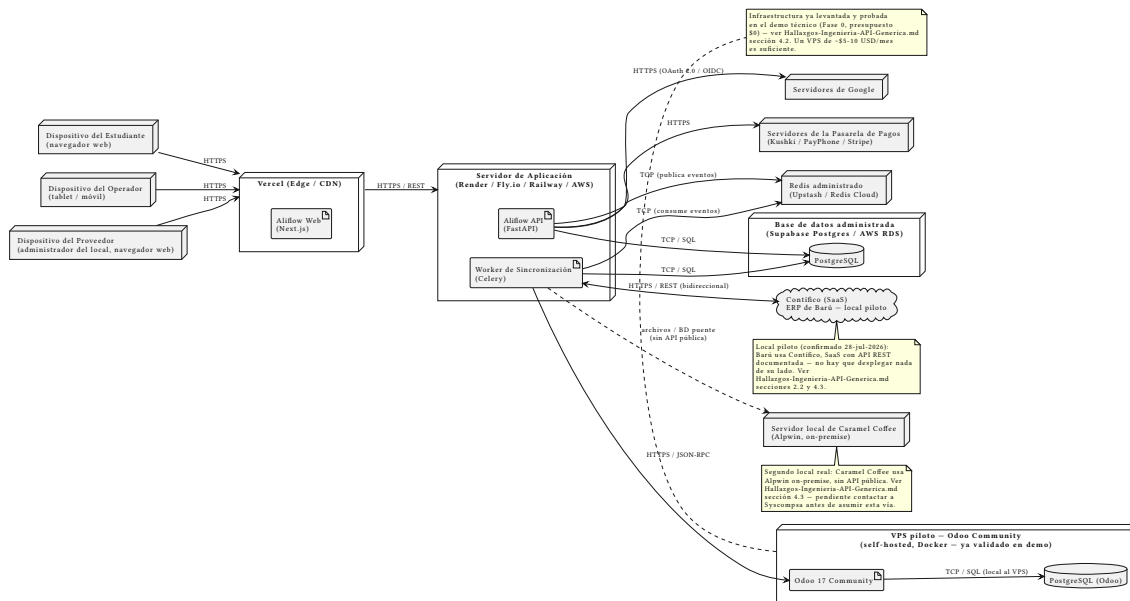


Figure 6: Diagrama de despliegue de Aliflow

Diagrama fuente: ../ ../uml/diagrama-despliegue.puml.

Este diagrama traduce el diagrama de componentes (../ ../uml/diagrama-componentes.puml) a infraestructura física/de hosting concreta, combinando tres fuentes reales:

1. Las opciones de despliegue investigadas previamente por el equipo (Vercel, Render/Fly.io/Railway/AWS, Supabase/AWS RDS).
2. La infraestructura que **ya se levantó y probó** en el demo técnico con Odoo Community (.../.../Hallazgos-Ingenieria-API-Generica.md sección 4.2).
3. La confirmación de Negocios (28-jul-2026) de que **cada local trae su propio ERP**: Barú usa Contífico (SaaS, con API REST) y Caramel Coffee usa Alpwin (on-premise, sin API pública) — sección 4.3 del mismo documento.

Nodos

Nodo	Contenido	Notas
Dispositivos (Estudiante, Operador, Proveedor)	Navegador web / tablet	Una sola aplicación web (no apps nativas separadas); el flujo ya documentaba que el Operador usa “interfaz simplificada... móvil o tablet” (op-1).
Vercel (Edge/CDN)	Aliflow Web (Next.js)	Hosting típico para Next.js — despliegue estático/edge con baja latencia.
Servidor de Aplicación (Render/Fly.io/Railway/AWS)	Aliflow API (FastAPI) + Worker de Sincronización (Celery)	Se muestran como dos artefactos en el mismo nodo lógico, aunque en producción normalmente correrían como dos servicios/procesos escalables por separado.
Redis administrado (Upstash/Redis Cloud)	—	Cache + broker de Celery (patrón Outbox).
Base de datos administrada (Supabase Postgres/AWS RDS)	PostgreSQL	Persistencia del dominio de Aliflow.
VPS piloto — Odoo Community	Odoo 17 Community + PostgreSQL (Odoo)	Esta es la única infraestructura de este diagrama que ya existe y fue probada realmente (docker-compose.yml en .../.../demo-odoo/) — no es una propuesta teórica.
Contífico (SaaS)	ERP de Barú, el local piloto	Fuera del control de Aliflow, pero no hay que desplegar nada: es un servicio en la nube con API REST documentada.
Servidor local de Caramel Coffee	Alpwin (on-premise)	Fuera del control de Aliflow — es la infraestructura de ese local, no la nuestra.
Servidores de Google / Pasarela de Pagos	—	Terceros, fuera de nuestro control.

Flujos de comunicación relevantes

- **Estudiante/Operador/Proveedor → Vercel → API**: todo el tráfico de usuario pasa por HTTPS/REST, sin excepción.
- **API/Worker → PostgreSQL**: persistencia síncrona vía SQL.
- **API → Redis (publica) / Worker → Redis (consume)**: implementación real del patrón Outbox — la API nunca llama directamente al ERP externo, solo encola el evento.

- **Worker → Odoo (JSON-RPC)**: el mismo protocolo ya usado y depurado en el demo (recordar el hallazgo de que hubo que usar JSON-RPC en vez de XML-RPC por el problema de serializar None).
- **Worker ↔ Contífico (HTTPS/REST, bidireccional)**: línea sólida — la API existe y está documentada; falta únicamente que Barú entregue las credenciales (riesgo R-01).
- **Worker → Alpwin (archivos/BD puente)**: línea punteada — a diferencia de las otras dos, esta vía **no está implementada ni confirmada todavía**; depende de lo que responda Syscompsa (pendiente en el checklist de ../../../../Hallazgos-Ingenieria-API-Generica.md).

Decisiones que quedan abiertas en este diagrama

1. **Proveedor de hosting final para la API** (Render vs. Fly.io vs. Railway vs. AWS) — no se ha decidido, solo se documentan como candidatas ya investigadas.
2. **Si el Worker corre como proceso separado o dentro del mismo contenedor que la API** — aquí se muestran como dos artefactos independientes (más correcto para escalar cada uno según su carga), pero es una decisión de implementación pendiente.
3. **La conexión real con Alpwin** — este diagrama asume la vía de archivos/BD puente como hipótesis de trabajo; podría cambiar completamente si Syscompsa ofrece algo distinto, o si el local que lo usa (Caramel Coffee) decide migrar (ver sección 4.3 del hallazgo de Ingeniería). Ya **no bloquea el piloto**, porque el local piloto es Barú con Contífico.
4. **El diagrama muestra dos ERP externos, pero el modelo admite N** — se dibujan Contífico y Alpwin porque son los dos locales confirmados; cada local nuevo agrega un nodo externo más, sin cambiar nada de la infraestructura de Aliflow.